1

Focus on High-Leverage Activities

In the span of three months, Quora's engineering team doubled. Our startup had an ambitious mission: to build a question-and-answer platform that would share and grow the world's knowledge. To achieve that internet-scale Library of Alexandria, we needed more engineers. And so, during the summer of 2012, 14 new engineering hires crammed into our small office on the second floor of 261 Hamilton Avenue in downtown Palo Alto, California. Even after construction crews knocked down three sets of walls to expand our office space, we could still barely accommodate everyone's desks.

I had met many times with Charlie Cheever, one of Quora's two cofounders, to prepare for the influx of people. I had volunteered to be directly responsible for onboarding new engineering hires, and so I needed to design the team's plan to ramp everyone up quickly. How would we ensure that the product wouldn't break all the time, especially since we deployed code to production multiple times a day? How would we keep code quality high with so many new people unfamiliar with the design and conventions in the codebase? How would we make sure new hires were productive and not blocked on what to do next? Figuring out these answers would be a marked departure from my regular duties of writing software, but I was excited because I knew that onboarding the new hires effectively would have a larger impact than any code I

could produce. It would directly affect how much work half of the engineering team would get done during their first few months.

I researched onboarding and mentoring programs at other companies and talked with their engineers to learn what worked and what didn't. With support from the rest of my team, I formalized a mentoring program at Quora for new engineering hires. Each new engineer was paired up with a mentor for two to three months, and the mentor was responsible for the new hire's success. Mentoring activities included everything from reviewing code, outlining technical skills to learn, and pair programming, to discussing engineering tradeoffs, explaining how to prioritize better, and offering guidance on how to work well with different team members. Mentors also planned a sequence of starter tasks and projects to increase the new hires' mastery of our systems.

To ensure that our hires started on a common foundation, I also created an onboarding program with a series of 10 recurring tech talks and 10 *codelabs*. Codelabs, an idea I borrowed from Google, were documents that explained why we designed core abstractions, described how to use them, walked through the relevant code, and provided exercises to validate understanding. Through tech talks and codelabs, we gave our hires an overview of the codebase and the engineering focus areas, and taught them how to use our development and debugging tools.

As a result of these efforts, many of our hires that summer were able to successfully deploy their first bug fix or small product feature to users by the end of their first week, and others deployed their first changes soon after. During those months, we launched a new Android application, an internal analytics system, better in-product recommendations, improvements to our product's news feed, and much more. Code quality stayed high, and mentors met regularly to share thoughts on how to make the onboarding process even smoother. Even after I moved on from the company, dozens of new engineers have continued to go through these onboarding and mentoring programs. [1] They were two of the highest-leverage investments that we made on the engineering team.

In this chapter, we'll define what leverage is and explain why it's the yardstick for measuring our effectiveness. Then we'll go over three ways of reasoning about how to increase the leverage of our activities. And finally, we'll dis-

cuss why directing energy toward our leverage points, as opposed to easy wins, is key to increasing our impact.

Use Leverage as Your Yardstick for Effectiveness

Why did Quora engineers spend so much energy on mentoring and onboarding new engineers? We all had hundreds of other things that we could have worked on—ideas to prototype, features to build, products to launch, bugs to fix, and teams to run. Why didn't we focus on these instead? More generally, given the hundreds or more tasks that we all could be doing in our jobs right now, how should we decide what to actually work on in order to more effectively achieve our goals?

The key to answering these questions and prioritizing these different activities is assessing their *leverage*. *Leverage* is defined by a simple equation. It's the value, or impact, produced per time invested:

$$\text{Leverage} = \frac{\text{Impact Produced}}{\text{Time Invested}}$$

Put another way, leverage is the return on investment (ROI) for the effort that's put in. Effective engineers aren't the ones trying to get more things done by working more hours. They're the ones who get things done efficiently—and who focus their limited time on the tasks that produce the most value. They try to increase the numerator in that equation while keeping the denominator small. Leverage, therefore, is the yardstick for measuring how effective your activities are.

Leverage is critical because time is your most limited resource. Unlike other resources, time cannot be stored, extended, or replaced.[2] The limitations of time are inescapable, regardless of your goals. You might be a product engineer deciding what to tackle to maximize your impact on users. Or you might be an infrastructure engineer figuring out what scaling issue to focus on next. You might be a workaholic who loves to code 60 to 70 hours per week at the office. Or you might be a subscriber to Timothy Ferris's *4-Hour Work Week* philoso-

phy and only want to put in the minimal hours per week required to sustain your lifestyle business. No matter who you are, at some point in your career you'll realize that there's more work to be done than time available, and you'll need to start prioritizing.

Another way of thinking about leverage is the commonly-mentioned Pareto principle, or 80–20 rule—the notion that for many activities, 80% of the impact comes from 20% of the work. [3] That 20% comprises the high-leverage activities, activities that produce a disproportionately high impact for a relatively small time investment.

Effective engineers use this simple equation as their central, guiding metric for determining how and where to spend their time. Greek mathematician and engineer Archimedes once declared, "Give me a place to stand, and a lever long enough, and I shall move the world." [4] It can be hard to move a huge boulder by yourself, but with a powerful enough lever, you can move almost anything. High-leverage activities behave similarly, letting you amplify your limited time and effort to produce much more impact.

Based on the principle of leverage, it's clear why our engineering team focused on mentoring and training new engineers. Mentoring is a prime example of a high-ROI activity. In a single year, a typical engineer works between 1,880 and 2,820 hours. [5] Devoting even 1 hour every day for the first month (20 hours) to mentor or train a new hire may seem like a large investment; yet it represents only about 1% of the total time the new hire will spend working during her first year. Creating reusable resources like codelabs paid off even higher dividends and required little upkeep after an upfront investment.

Moreover, that 1% time investment can have an outsized influence on the productivity and effectiveness of the other 99% of work hours. Pointing out a useful UNIX command could save minutes or hours on basic tasks. Walking through debugging tools could drastically reduce development time for every new feature. Reviewing code early and thoroughly to catch common errors could remove the need to re-address similar issues later and prevent bad habits from forming. Teaching a new hire how to prioritize different projects to complete or skills to learn could easily increase his productivity. Planning good

starter projects that teach core abstractions and fundamental concepts could improve her software designs and reduce future maintenance requirements.

Since the success of our startup depended more on the success of the entire team than on what any one engineer could accomplish, investing in programs to ramp up new engineers as quickly and as seamlessly as possible was one of the highest-leverage activities that we could have done.

Increase Your Leverage in Three Ways

In his book *High Output Management*, Former Intel CEO Andrew Grove explains that by definition, your overall leverage—the amount of value that you produce per unit time—can only be increased in three ways: [6]

1. By reducing the time it takes to complete a certain activity.
2. By increasing the output of a particular activity.
3. By shifting to higher-leverage activities.

These three ways naturally translate into three questions we can ask ourselves about any activity we're working on:

1. How can I complete this activity in a shorter amount of time?
2. How can I increase the value produced by this activity?
3. Is there something else that I could spend my time on that would produce more value?

Your output as an engineer is measured in a variety of ways, including the numbers of products launched, bugs fixed, users acquired, and engineers hired, along with ranking quality improved, revenue generated, and many other metrics. Your total output is the sum of the output of your individual activities. In a typical work day, these activities might include attending meetings, responding to emails, investigating bugs, refactoring old code, developing new features, reviewing changes, monitoring metrics, maintaining production systems, interviewing potential hires, and more.

However, spending your work day going through the motions of these different activities does not necessarily mean that you're producing value the entire time. As Figure 1 shows, each individual daily activity has its own leverage,

measured by the activity's output divided by the time it takes to complete that activity. Some activities, like implementing a feature request, learning a new testing framework, or fixing an important bug, have high leverage; others, like surfing the web or responding to email, might take up just as much time but have lower leverage because they don't generate as much value.

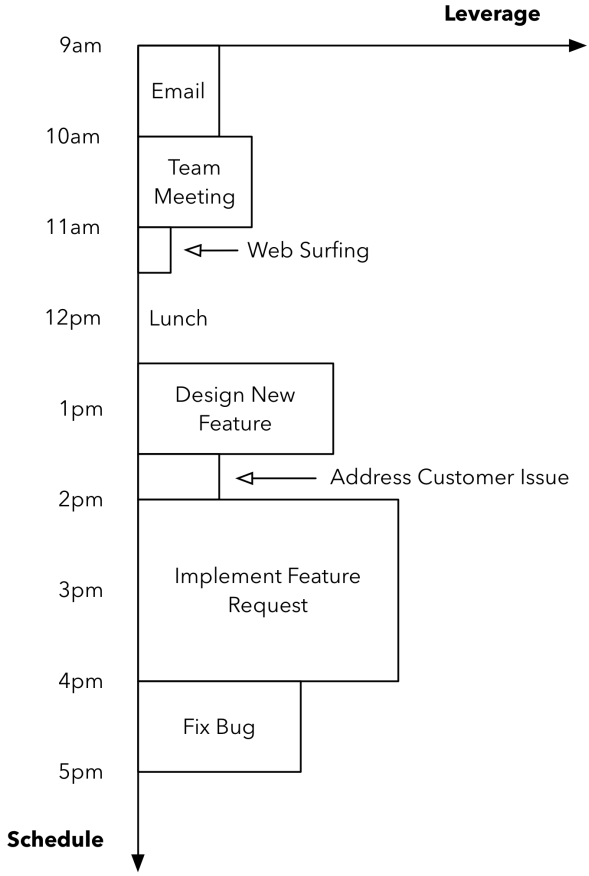


Figure 1: Leverage of different activities during a typical workday.

To increase the leverage of each activity, ask yourself the previous three questions, each of which leads to a different avenue of potential improvements. For example, you might have a one-hour meeting that you've scheduled with your

team to review their progress on a project. You can increase the meeting's leverage by:

1. Defaulting to a half-hour meeting instead of a one-hour meeting to get the same amount done in less time.
2. Preparing an agenda and a set of goals for the meeting and circulating them to attendees beforehand so that the meeting is focused and more productive.
3. If an in-person discussion isn't actually necessary, replacing the meeting with a email discussion and spending the time building an important feature instead.

Or perhaps you're a product engineer, ready to start working on a new customer-facing feature for your company's flagship product. You might increase the leverage of your development time by:

1. Automating parts of the development or testing process that have thus far been done manually, so that you can iterate more quickly.
2. Prioritizing tasks based on how critical they are for launch so that you maximize the value of what you finally ship.
3. Talking with the customer support team to gain insights into the customers' biggest pain points, and using that knowledge to understand whether there's another feature you could be working on that would produce even more value with less development effort.

Or suppose that you're a performance engineer identifying and fixing the bottlenecks in a web application. The application might slow down as the product teams launch new products and features, and it's your job to ensure that it stays fast. Some approaches you might consider to increase your leverage include:

1. Learning to effectively use a profiling tool so that you can reduce the time it takes to identify each bottleneck.
2. Measuring both the performance and visit frequency of each web page so that you can address the bottlenecks that affect the most traffic first, thereby increasing the impact of each fix you make.

3. Working with product teams to design performant software from the outset, so that application speed is prioritized as a feature during product development rather than treated as a bug to be fixed.

As these examples illustrate, for any given activity, there are three approaches you can take to increase the leverage of your time spent. When you successfully shorten the time required for an activity, increase its impact, or shift to a higher-leverage activity, you become a more effective engineer.

Direct Energy Toward Leverage Points, Not Just Easy Wins

We have a limited amount of time and a large number of possible activities. As you make your way through this book, constantly keep one lesson in mind: *focus on high-leverage activities.* This is the single most valuable lesson that I've learned in my professional life.

Don't confuse high-leverage activities with easy wins, however. Just as a lever lets you apply a small amount of force over a long distance to generate a much larger force, so too do many high-leverage activities require consistent applications of effort over long time periods to achieve high impact. Yishan Wong, an early director of engineering at Facebook and a former CEO of Reddit, shared a story with me of his proudest accomplishment at Facebook that underscores this point. [7]

Facebook has a strong hiring culture. Employees view themselves as the guardians of high standards, and hiring is a top priority for both managers and engineers. But it wasn't always this way. When Wong first entered into a management position at Facebook in late 2006, engineers viewed recruiting and interviews as distractions from getting their work done [8]—an attitude that's common to engineers at many companies. Everyone knew on some level that hiring was important, but translating it into action was another matter.

Wong had to gradually apply pressure to change the prevailing mindset to one where people considered the hiring process an art form to be mastered. When engineers asked him how the company would ensure that the people being hired were strong engineers, Wong would tell them that this was *their* job. And because hiring was a top priority, engineers didn't skip interviews to do

other work. They were expected to submit feedback immediately, as opposed to deferring it for hours or days. When recruiters needed to schedule interviews with candidates, Wong would push them to schedule the conversations in the "first humanly-possible time slot." [9] Can't meet until tomorrow? How about 8am? Not free until the afternoon? What about 1pm? Wong built a culture where even rejected candidates would leave interviews wanting to work at Facebook. And over his four years at the company, an obsession with speed and quality in hiring became one of Facebook's competitive advantages. While slower-moving companies dilly-dallied, Facebook closed candidates.

Building a strong hiring culture wasn't a quick and easy fix; it was a high-leverage activity that required consistent effort over many years. Once strong people joined the company, however, it became easier to attract more strong people. It's clear that Facebook could not have become a massively successful $220+ billion company with over 9,000 employees without its emphasis on the hiring process. [10]

Our own work might not be on as grand a scale, but the concept of leverage is as powerful of a framework for us as it is for Facebook. Moreover, leverage is used not just by effective engineers, but also by the world's most successful people. Bill Gates, for example, after retiring as the CEO of Microsoft, started focusing his time and energy on how to best invest his billions of dollars in philanthropy. While the $40.2 billion managed by the Bill & Melinda Gates Foundation may be a lot of money, it's still not nearly enough to solve all of the world's problems. [11] "[I]n a global economy worth tens of trillions of dollars, any philanthropic effort is relatively small," writes Gates in a November 2013 *Wired* essay. "If you want to have a big impact, you need a leverage point—a way to put in a dollar of funding or an hour of effort and benefit society by a hundred or a thousand times as much." For Gates, those leverage points include funding measles and malaria vaccines that cost less than 25 cents each but that save millions of lives. [12]

Similarly, engineering has its own set of leverage points. This book won't be a substitute for mindful reflection, but it can help shortcut your path to finding these points. In each subsequent chapter, you'll find a high-leverage habit of effective engineers—reinforced by scientific research, stories from industry,

and concrete examples. You'll read about why the leverage of each habit justifies its time investment, and you'll also find concrete and actionable tips for incorporating each habit into your own craft. Doubling down on these leverage points will help you transform the time and effort you spend as an engineer into meaningful impact.

Key Takeaways

- **Use leverage to measure your engineering effectiveness.** Focus on what generates the highest return on investment for your time spent.
- **Systematically increase the leverage of your time.** Find ways to get an activity done more quickly, to increase the impact of an activity, or to shift to activities with higher leverage.
- **Focus your effort on leverage points.** Time is your most limited asset. Identify the habits that produce disproportionately high impact for the time you invest.